Week 1: What is Design Quality?

Analysis and Assessment of Design Quality
Анализ и оценка качества проектирования

Today's learning goals

By the end of this lecture you should be able to:

1. Distinguish evaluating design quality from inventing architectures
2. Define design quality as purpose fit + attributes + structure + justified trade-offs
3. Name artifacts that are evaluable vs decorative
4. Map stakeholder viewpoints that pressure quality differently

Who this course is for

- Master's students in Software Engineering
- You already sketch modules, APIs, and deployment ideas
- Goal: become a reliable evaluator of design quality

MSc note

We stay practical: scenarios, smells, checklists, light metrics, rubrics, reports. No deep ATAM mathematics; methods must transfer to industry design reviews.

Two complementary skills

Design / architecture

Propose structure.
Choose styles and boundaries.
Write decisions (ADR-lite).

Quality assessment (this course)

Ask: how good, for whom, under what load?
Surface smells and risks.
Score with rubrics; recommend next steps.

Strong engineers do both—but confuse them when they "fix" by redrawing.

Evaluation is not rewriting

A reviewer's job:

1. Understand the author's intent and constraints
2. Test claims against scenarios and structure
3. Leave actionable findings with severity
4. Separate preference from evidence

Common pitfall

Replacing the author's design with yours and calling it a "review." That is co-design—useful sometimes, but not assessment.

Quiz 1 — Role of the evaluator

Quiz

A teammate's layered sketch shows the UI calling the database directly. Which response is the best evaluation move first?

1. Silently redraw the layers "the right way" and present your version
2. Ask which quality scenarios (if any) the shortcut is meant to serve, then cite layering risk against those scenarios
3. Mark it wrong because Clean Architecture forbids it in all contexts
4. Ignore it—diagrams are only decorative

Pause — discuss with a neighbour or answer privately before the next slide.

Answer — Quiz 1

Answer

2. Start from intent and scenarios, then evidence the risk. Redrawing (1) skips assessment; dogma (3) ignores context; (4) abandons evaluability.

Working definition

Design quality (for this course) is the degree to which a software design:

1. Fits stated purpose and constraints
2. Satisfies prioritized quality attributes under realistic scenarios
3. Exhibits structural clarity (boundaries, ownership, dependencies)
4. Makes trade-offs explicit and justified

Pretty diagrams help communication; they are not quality by themselves.

Fitness for purpose vs elegance

- A “beautiful” hexagonal design can still fail latency or cost constraints
- A “boring” modular monolith can be high quality for the brief
- Quality is relative to goals, load, team, and time horizon

Try this

For your last project: name one elegant choice that was low value for the brief, and one ugly choice that was high value.

Quiz 2 — What counts as quality

Quiz

Which claim is strongest as a design-quality judgment?

1. "The colour palette on the diagram is inconsistent."
2. "I prefer microservices; this monolith looks outdated."
3. "Given the maintainability scenario (change payment rules weekly), three modules must change together—evidence of a cohesion/coupling issue."
4. "The design uses seven fashionable pattern names."

Pause — discuss with a neighbour or answer privately before the next slide.

Answer — Quiz 2

Answer

3. Ties a scenario to structural evidence. (1) is presentation nits; (2) is preference; (4) is jargon density.

Artifacts we typically review

- Context / container / module views (C4-lite OK)
- Layering or package dependency sketches
- API surface (resources, commands, error model)
- Short decision notes (context, options, decision, consequences)
- Quality scenarios and non-goals

Code snippets are optional evidence, not the only artifact.

What makes an artifact evaluable?

Evaluable	Hard to judge
Named boundaries	Unlabelled boxes
Explicit dependencies	“Magic” middle tier
Stated assumptions	No load / failure story
Linked to scenarios	Goals only as slogans

Part of quality assessment is judging the adequacy of the design documentation.

Viewpoints that pull quality apart

Viewpoint	Often prioritises
End users	Usability, latency, correctness
Maintainers	Modularity, testability, clarity
Operators / SRE	Reliability, observability, deployability
Security	Isolation, least privilege, auditability
Product / business	Time-to-market, cost, feature velocity

A “perfect” design for one viewpoint often harms another—reviews must surface that.

Conflicting pressures (mini case)

Brief: campus registration system, peak at semester start.

- Product wants new waitlist rules each term (changeability)
- Ops wants zero downtime during peaks (availability)
- Security wants strict role checks on every write

Remember

Your evaluation should name which attributes win under stated constraints—not pretend all can be maximised.

Quiz 3 — Stakeholder conflict

Quiz

Ops demands a shared "god" admin service for faster incident response. Security objects. As evaluator, which statement is most professional?

1. "Security always wins; reject the admin service."
2. "Ship the god service; incidents matter more than theory."
3. "Document the trade-off: operational speed vs blast radius; require compensating controls (audit, MFA, scoped roles) if accepted."
4. "Average the two opinions into a medium-sized service."

Pause — discuss with a neighbour or answer privately before the next slide.

Answer — Quiz 3

Answer

3. Evaluation names the conflict and conditions of acceptance. (1)/(2) pick sides without evidence structure; (4) often creates the worst of both.

Crowded ≠ good

Anti-patterns in “impressive” packs:

- Wallpaper diagrams with no legend or trust boundaries
- Pattern name-dropping without decision consequences
- Metrics without interpretation
- Goals like “scalable and secure” with no scenarios

Try this

Grade a public architecture blog post: can you extract three testable scenarios?

Quality of the evaluation itself

A strong MSc evaluation report:

1. States brief, constraints, and non-goals
2. Lists prioritized attributes + scenarios
3. Shows evidence on artifacts (not vibe)
4. Scores with a rubric and explains disagreements
5. Ends with risks and next steps

We build these skills across Weeks 2–8.

Quiz 4 — Evaluability of artifacts

Quiz

Which design pack is most ready for a rigorous review?

1. A single slide of 40 unnamed boxes and arrows
2. Context + container views, three quality scenarios with response measures, and one ADR on monolith vs split services
3. A list of technologies (Kafka, K8s, GraphQL) with no structure
4. Only source code, no views or scenarios

Pause — discuss with a neighbour or answer privately before the next slide.

Answer — Quiz 4

Answer

2. Views + scenarios + decision rationale enable evidence-based critique. (1)/(3) lack evaluable structure; (4) can support review but is costly without views—usually insufficient alone for design-quality assessment.

Key takeaways

- This course trains assessment, not architecture invention
- Design quality = purpose + attributes + clarity + justified trade-offs
- Stakeholders conflict; good reviews surface conflicts
- Artifacts must be evaluable; decoration is not evidence

Practice today (2 h)

Markdown handout: 01-week01-practice.md

- Rank blurbs for evaluability
- Separate taste from evidence-based claims
- Stakeholder / conflict map for a short brief

Remember

Done checklist at the end of the handout—bring it to the lecturer checkpoint.

Next week

Quality attributes & scenarios

Structured scenarios (stimulus → response measure), attribute conflicts, non-goals.